Be Strong! | UNIVERSITAS LAMPUNG
Berintegritas Anti Korupsi

# MODUL PRAKTIKUM PEMROGRAMAN WEB




Laboratorium Teknik Komputer
Program Studi Teknik Informatika
Jurusan Teknik Elektro
Fakultas Teknik, Universitas Lampung
Jl. Prof. Sumantri Brojonegoro Ged H Fakultas Teknik
Bandar Lampung 35145

# JavaScript dan Manipulasi Document Object Model (DOM)

## 1. Tinjauan Pustaka

### 1.1 Arsitektur Web Modern dan Peran Lapisan Perilaku (Behavior Layer) JavaScript

Teknologi antarmuka web modern dibangun di atas triad pemisahan kepentingan (separation of concerns): HTML (HyperText Markup Language) berfungsi sebagai lapisan struktur dan konten semantik, CSS (Cascading Style Sheets) sebagai lapisan presentasi estetika visual, serta JavaScript sebagai lapisan perilaku (behavior layer) dan interaktivitas logis. Tanpa JavaScript, aplikasi web hanyalah dokumen hiperteks statis pasif; setiap interaksi pengguna seperti pengiriman formulir atau pembaruan data mengharuskan peramban memuat ulang seluruh halaman (full page refresh) melalui siklus HTTP Request-Response bolak-balik ke server web.

JavaScript bertindak sebagai motor penggerak komputasi di sisi peramban pengguna (client-side execution engine). Dengan JavaScript, aplikasi web dapat memberikan respon interaktif seketika terhadap tindakan pengguna tanpa latensi jaringan: memvalidasi isian formulir secara instan, memanipulasi susunan elemen visual secara dinamis, melakukan penyaringan dan pencarian data real-time, serta memperbarui nilai statistik dalam memori. Eksekusi ini dijalankan langsung oleh mesin peramban modern (seperti V8 Engine pada Google Chrome dan Microsoft Edge, SpiderMonkey pada Mozilla Firefox, dan JavaScriptCore pada Safari) yang mengombinasikan Call Stack eksekusi tunggal, Web APIs, Callback Queue, dan mekanisme Event Loop non-blocking.

### 1.2 Sejarah Perkembangan JavaScript dan Standar ECMAScript (ES6+)

JavaScript pertama kali diciptakan oleh Brendan Eich pada Mei 1995 hanya dalam waktu 10 hari saat bekerja di Netscape Communications Corporation. Semula dinamakan Mocha, lalu diubah menjadi LiveScript, dan akhirnya dipasarkan dengan nama JavaScript untuk memanfaatkan popularitas bahasa Java pada masa itu. Untuk mencegah fragmentasi bahasa antar-peramban saingan (khususnya Internet Explorer dengan JScript), Netscape menyerahkan spesifikasi bahasa tersebut kepada Ecma International pada November 1996, yang kemudian melahirkan standar resmi bernomor ECMA-262 bernama ECMAScript.

Perkembangan paling revolusioner dalam sejarah JavaScript terjadi pada Juni 2015 melalui peluncuran standar ECMAScript edisi ke-6, yang dikenal luas sebagai ES6 atau ECMAScript 2015. Pembaruan ini mentransformasikan JavaScript dari sekadar bahasa skrip ringan pembuat animasi menjadi bahasa pemrograman modern berskala enterprise. Fitur-fitur esensial ES6 meliputi: pendeklarasian variabel berlingkup blok (let dan const), fungsi panah (Arrow Functions), string berinterpolasi ekspresi (Template Literals `...`), sintaks destrukturisasi (Destructuring Assignment), parameter default, penyebaran array (Spread Operator ...), serta struktur data modern. Sejak 2015, komite TC39 mengadopsi rilis tahunan (ES2016 hingga ECMAScript modern) untuk menjamin inovasi berkelanjutan.

### 1.3 Sintaks Dasar, Variabel, Tipe Data, dan Struktur Objek/Array

Pendeklarasian variabel pada JavaScript modern wajib meninggalkan kata kunci klasik var karena memiliki kelemahan scope berbasis fungsi (function-scoped) dan perilaku pengangkatan variabel yang rawan memicu bug (hoisting). Standar industri modern mewajibkan penggunaan let dan const yang memiliki batasan lingkup blok kurung kurawal (block-scoped). Kata kunci const digunakan untuk mendeklarasikan identifier yang referensi memorinya bersifat tetap (immutable binding), sedangkan let digunakan untuk variabel yang nilainya akan mengalami mutasi atau penugasan ulang.

JavaScript membedakan data menjadi dua kelompok utama: Tipe Data Primitif (Primitive Types) yang disimpan langsung berdasarkan nilai (pass-by-value) seperti string, number, boolean, null, undefined, symbol, dan bigint; serta Tipe Data Referensi (Reference Types) berupa Objek (Objects) dan Array yang disimpan berdasarkan referensi memori (pass-by-reference). Dalam arsitektur aplikasi web interaktif seperti Sistem Informasi Data Mahasiswa (SIMA), struktur data yang paling fundamental adalah Array of Objects, yaitu sebuah larik (array) yang menampung sekumpulan objek data mahasiswa terstruktur di mana setiap objek menyimpan pasangan kunci dan nilai (key-value pairs) seperti npm, nama, program_studi, angkatan, dan email.

**Tabel 1 Komparasi Deklarasi Variabel JavaScript Modern:** Ringkasan karakteristik pembeda antara var, let, dan const:

| Kata Kunci | Cakupan Lingkup (Scope) | Penugasan Ulang (Reassign) | Redeklarasi | Perilaku Hoisting | Rekomendasi Penggunaan |
|---|---|---|---|---|---|
| var | Function Scope / Global | Diizinkan (Mutable) | Diizinkan (Rawan Bug) | Diangkat dengan inisialisasi undefined | Usang (Dilarang dalam modern JS). |
| let | Block Scope { ... } | Diizinkan (Mutable) | Dilarang dalam scope sama | Temporal Dead Zone (TDZ) | Gunakan jika nilai akan berubah. |
| const | Block Scope { ... } | Dilarang (Immutable binding) | Dilarang dalam scope sama | Temporal Dead Zone (TDZ) | Pilihan default untuk seluruh deklarasi. |

### 1.4 Konsep Dasar Document Object Model (DOM) dan Hierarki Pohon Dokumen (DOM Tree)

Document Object Model (DOM) adalah antarmuka pemrograman berorientasi objek (W3C standard API) yang merepresentasikan dokumen HTML sebagai struktur pohon berhierarki (DOM Tree). Saat peramban web memuat berkas index.html, mesin perender (rendering engine) mem-parsing baris-baris markup HTML dan membangun representasi objek pohon di dalam memori. Di dalam pohon ini, setiap tag HTML, atribut elemen, dan penggalan teks ditransformasikan menjadi Node objek mandiri yang dapat diakses, dibaca, dan dimanipulasi secara dinamis menggunakan skrip JavaScript.

Titik masuk paling puncak dalam hierarki peramban adalah objek global window, yang mewakili jendela peramban tempat dokumen ditampilkan. Di dalam objek window terdapat objek document yang merupakan representasi langsung dari seluruh pohon dokumen web yang aktif saat ini. Penting untuk dipahami bahwa kode sumber HTML (HTML source code yang ditulis pengembang) tidak selalu identik dengan DOM yang sedang berjalan di layar: DOM bersifat

hidup (live representation) dan senantiasa berubah secara dinamis setiap kali skrip JavaScript menyisipkan baris tabel baru, menghapus baris data, atau mengubah atribut class.

**Tabel 2 Hirarki dan Tipe Node Pokok dalam DOM W3C:** Hierarki tipe node pokok dalam struktur pohon DOM:

| Tipe Node | Contoh Representasi | Deskripsi & Peran Arsitektural | Metode Inspeksi |
|---|---|---|---|
| Document Node | window.document | Akar tertinggi pohon dokumen web; titik awal seluruh operasi seleksi. | document.nodeType === 9 |
| Element Node | <body>, <table>, <tr>, <button> | Tag HTML yang membungkus konten dan mendefinisikan struktur antarmuka. | node.nodeType === 1 |
| Text Node | "Ahmad Fauzi", "Teknik Informatika" | Konten teks aktual yang terletak di antara tag pembuka dan penutup elemen. | node.nodeType === 3 |
| Attribute Node | class="table", id="npm", href="..." | Pasangan kunci-nilai metadata yang melekat langsung pada sebuah elemen. | node.attributes / getAttribute() |
| Comment Node | <!-- Komentar HTML --> | Catatan pengembang yang diabaikan perender visual namun ada di pohon DOM. | node.nodeType === 8 |

### 1.5 Metode Seleksi Elemen DOM Modern

Sebelum skrip JavaScript dapat berinteraksi dengan elemen web, peramban harus menemukan dan memilih node elemen target dari pohon DOM. DOM API menyediakan berbagai metode seleksi, mulai dari metode klasik berbasis pencarian indeks tunggal hingga metode modern berbasis CSS Selector yang sangat fleksibel.

Metode klasik seperti document.getElementById() adalah metode tercepat dan paling efisien untuk menemukan satu elemen spesifik berdasarkan atribut ID uniknya. Namun, untuk seleksi elemen yang kompleks atau berdasarkan pola kelas dan relasi hierarki, pengembang modern mengandalkan document.querySelector() dan document.querySelectorAll(). Metode querySelector() mengembalikan node elemen pertama yang cocok dengan pemilih CSS yang ditentukan, sedangkan querySelectorAll() mengembalikan seluruh elemen yang cocok dalam bentuk NodeList statis. NodeList statis memiliki keunggulan dibandingkan HTMLCollection klasik karena tidak rentan terhadap mutasi DOM tak terduga serta langsung mendukung metode iterasi bawaan seperti .forEach().

**Tabel 3 Komparasi Metode Seleksi Elemen DOM:** Perbandingan metode seleksi elemen DOM pada JavaScript:

| **Metode Seleksi** | **Parameter Masukan** | **Nilai Kembalian** | **Tipe Koleksi** | **Kecepatan Relatif** |
|---|---|---|---|---|
| getElementById() | "id-elemen" | Elemen tunggal / null | Bukan koleksi (Direct Node) | Sangat Cepat (Optimal) |
| querySelector() | ".kelas" / "#id" / "tag" | Elemen pertama yang cocok / null | Bukan koleksi (First Match) | Cepat (CSS Selector Engine) |
| querySelectorAll() | ".kelas" / "table tr td" | Kumpulan elemen cocok | NodeList Statis (mendukung forEach) | Sedang (Pencarian pohon menyeluruh) |
| getElementsByClassName() | "nama-kelas" | Kumpulan elemen cocok | HTMLCollection Hidup (Live Collection) | Cepat (Pencarian spesifik kelas) |

| | | | | |
|---|---|---|---|---|
| getElementsByTagName() | "tr" / "input" | Kumpulan elemen cocok | HTMLCollection Hidup (Live Collection) | Cepat (Pencarian spesifik tag) |

### 1.6 Manipulasi Konten Teks, Struktur HTML, dan Atribut Elemen

Setelah elemen target terpilih, skrip dapat mengubah isi konten, atribut, maupun kelas gayanya secara dinamis. Terdapat tiga properti utama untuk memanipulasi konten: textContent, innerText, dan innerHTML. Properti textContent membaca atau menulis teks mentah apa adanya di dalam node elemen tanpa memproses tag HTML; properti ini sangat cepat dan aman dari celah keamanan. Sebaliknya, properti innerText memperhitungkan gaya CSS (misalnya mengabaikan teks yang disembunyikan dengan display: none) sehingga memicu operasi layout reflow peramban. Properti innerHTML mem-parsing string masukan sebagai elemen markup HTML nyata, memungkinkan penyisipan baris tag secara instan.

Penggunaan innerHTML harus dilakukan dengan penuh kehati-hatian: jika pengembang menyisipkan data mentah dari masukan pengguna langsung ke dalam innerHTML tanpa sanitasi, aplikasi menjadi rentan terhadap serangan XSS (Cross-Site Scripting). Untuk memanipulasi atribut elemen, digunakan metode getAttribute('nama'), setAttribute('nama', 'nilai'), dan removeAttribute('nama'). Untuk manipulasi kelas visual, properti classList menyediakan metode modern yang elegan: classList.add('show'), classList.remove('hidden'), classList.toggle('active'), serta classList.contains('selected') tanpa perlu memanipulasi string className secara manual.

**Tabel 4 Komparasi Properti Manipulasi Konten DOM:** Perbandingan karakteristik properti manipulasi isi konten:

| Properti DOM | Pemrosesan HTML | Pertimbangan Reflow CSS | Resiko Keamanan (XSS) | Skenario Penggunaan Utama |
|---|---|---|---|---|
| textContent | Tidak (Teks mentah aman) | Tidak (Sangat cepat) | Aman (Kebal injeksi skrip) | Mengubah label, counter, dan notifikasi teks. |
| innerText | Tidak (Teks kasat mata) | Ya (Memperhitungkan styling CSS) | Aman (Teks dievaluasi) | Membaca teks yang benar-benar tampak di layar. |

| | | | | |
|---|---|---|---|---|
| innerHTML | Ya (String diparsing jadi tag) | Ya (Membangun ulang sub-pohon DOM) | Tinggi jika input tidak disanitasi | Membuat baris tabel dinamis dengan markup terformat. |

### 1.7 Pembuatan, Penyisipan, dan Penghapusan Node DOM Dinamis

Selain memodifikasi konten elemen yang sudah ada, aplikasi web dinamis sering kali dituntut untuk membuat elemen antarmuka baru dari nol atau menghapus elemen lama dari layar. Pola pembuatan node DOM secara prosedural menggunakan metode document.createElement('tagName') untuk mengalokasikan memori bagi elemen baru (misalnya membuat elemen baris <tr> baru). Karakteristik dan atribut elemen baru tersebut kemudian dikonfigurasi menggunakan properti objek (misalnya mengisi .innerHTML atau mengatur .className), sebelum akhirnya dicangkokkan ke dalam dokumen aktif menggunakan method parentElement.appendChild(childNode) atau insertBefore().

Untuk menghapus elemen dari dokumen, metode modern node.remove() memungkinkan elemen membuang dirinya sendiri secara instan dari pohon DOM tanpa perlu menelusuri elemen induknya. Alternatif klasik yang masih didukung adalah parentNode.removeChild(childNode). Dalam rekayasa performa web, manipulasi DOM yang dilakukan berulang-ulang dalam satu perulangan besar (misalnya menambahkan 100 baris tabel satu demi satu) dapat menyebabkan kemacetan peramban akibat pemicuan siklus Reflow (penghitungan ulang posisi dan dimensi fisik elemen) dan Repaint (penggambaran ulang piksel ke monitor). Untuk menjaga efisiensi rendering, pengembang disarankan merakit string HTML utuh sebelum memasukkannya sekaligus ke DOM, atau memanfaatkan DocumentFragment sebagai kontainer penampung sementara di memori.

### 1.8 Arsitektur Berbasis Peristiwa (Event-Driven) dan Event Handling

Aplikasi web berinteraksi dengan manusia melalui arsitektur berbasis peristiwa (Event-Driven Architecture). Peramban web secara kontinu memantau interaksi pengguna di atas jendela aplikasi: penekanan tombol mouse (click), penekanan tombol papan ketik (keydown, input),

pengiriman data formulir (submit), pergerakan kursor (mouseover), hingga selesainya proses parsing dokumen (DOMContentLoaded). Ketika suatu peristiwa terjadi pada sebuah elemen, peramban memicu objek Peristiwa (Event Object) dan memanggil fungsi penangan (Event Listener / Callback Function) yang telah didaftarkan pada elemen bersangkutan.

Standar resmi W3C mewajibkan registrasi penangan peristiwa menggunakan metode target.addEventListener(event, callback, useCapture). Metode ini jauh lebih unggul dibandingkan atribut inline HTML klasik (seperti onclick="...") karena mendukung pemasangan lebih dari satu listener pada satu elemen, mendukung pemisahan kode secara bersih, serta memiliki akses penuh ke objek Event. Objek Event menyediakan informasi krusial mengenai konteks interaksi: event.target merujuk ke elemen fisik yang memicu kejadian, event.preventDefault() berfungsi membatalkan perilaku bawaan peramban (misalnya mencegah browser melakukan refresh saat formulir disubmit), dan event.stopPropagation() menghentikan perambatan peristiwa ke elemen induk di sepanjang hierarki pohon DOM (Event Bubbling).

**Tabel 5 Peristiwa (Events) Kunci dalam Antarmuka Web Interaktif:** Kategori peristiwa interaktif kunci yang digunakan dalam antarmuka web SIMA:

| **Nama Event** | **Kategori** | **Pemicu Interaksi Pengguna** | **Peran Kunci dalam Aplikasi SIMA** |
|---|---|---|---|
| DOMContentLoaded | Siklus Hidup Dokumen | Pohon DOM selesai diparsing lengkap oleh browser. | Menjalankan inisialisasi renderTable() awal dan alert box. |
| submit | Formulir | Pengguna mengklik tombol submit atau menekan Enter di form. | Memicu validasi data klien dan penambahan mahasiswa baru. |
| click | Perangkat Mouse | Pengguna mengklik tombol atau tautan aksi. | Memicu fungsi hapusMahasiswa() dan konfirmasi dialog. |

| | | | |
|---|---|---|---|
| input | Papan Ketik / Kontrol | Nilai pada elemen <input> mengalami perubahan seketika. | Memicu fungsi pencarian langsung (Live Search Filter). |
| reset | Formulir | Pengguna mengklik tombol bersihkan/reset form. | Mengembalikan seluruh isian formulir ke kondisi kosong semula. |

**1.9 Validasi Formulir di Sisi Klien (Client-Side Validation) dan Pola Regular Expression (RegEx)**

Validasi formulir di sisi klien (Client-Side Form Validation) adalah lapisan pertahanan pertama dalam siklus masukan data aplikasi web. Tujuan utamanya adalah memberikan umpan balik langsung kepada pengguna (instant user experience feedback) mengenai kesalahan ketik sebelum formulir diproses lebih jauh, menghemat sumber daya komputasi server, serta mencegah pengiriman data kosong atau korup. Meskipun sangat esensial bagi kenyamanan pengguna, validasi sisi klien tidak pernah boleh dijadikan satu-satunya sistem keamanan; validasi sisi server (Server-Side Validation) pada Praktikum 4 dan 5 tetap bersifat mutlak karena validasi JavaScript di peramban dapat dengan mudah dimanipulasi atau dimatikan oleh pengguna jahat.

Instrumen terkuat dalam validasi string di JavaScript adalah Regular Expression (RegEx). RegEx adalah urutan karakter khusus yang mendefinisikan pola pencarian (search pattern) untuk mencocokkan format teks. Dalam sistem SIMA, dua pola RegEx utama diterapkan: (1) Pola format NPM /^[0-9]{8,12}$/ yang menjamin string masukan hanya tersusun atas digit angka murni dengan panjang minimal 8 dan maksimal 12 digit; dan (2) Pola format alamat email /^[^\s@]+@[^\s@]+\.[^\s@]+$/ yang memastikan adanya nama kotak surat, simbol pemisah @, nama domain, titik domain, dan ketiadaan spasi kosong. Selain validasi format teks, fungsi validasi juga melakukan sanitasi spasi menggunakan .trim(), konversi tipe data angka dengan parseInt(), serta pengecekan logika bisnis mendalam berupa pencegahan duplikasi data NPM dalam memori menggunakan metode array .some().

**Tabel 6 Karakter Khusus & Pola Regular Expression Populer:** Karakter khusus dan pola Regular Expression populer untuk validasi:

| Karakter RegEx | Makna & Peran Pencocokan | Contoh Pola | Keterangan Fungsi |
|---|---|---|---|
| ^ dan $ | Penanda awal baris (^) dan akhir baris ($). | /^...$/ | Memastikan pencocokan tepat satu string utuh tanpa sisa. |
| [0-9] / \d | Mencocokkan satu karakter digit angka. | /^[0-9]+$/ | Hanya mengizinkan masukan angka numerik. |
| {min,max} | Kuantifier penentu batas jumlah pengulangan karakter. | /^[0-9]{8,12}$/ | Mewajibkan jumlah digit berada di antara rentang 8 s.d. 12. |
| [^\s@] | Karakter negasi: mencocokkan karakter apa pun selain spasi dan @. | /^[^\s@]+@.../ | Mencegah spasi kosong pada username dan domain email. |
| \. | Karakter titik literal (diberi escape backslash). | /\.[a-z]{2,}$/ | Mencocokkan tanda titik pemisah ekstensi domain web. |

### 1.10 Manajemen State Lokal (Single Source of Truth) dan Pemfilteran Data Real-Time (Live Search)

Arsitektur perangkat lunak frontend modern menganut prinsip Manajemen State (State Management) berbasis Single Source of Truth (Satu Sumber Kebenaran Tunggal). Dalam paradigma ini, data murni aplikasi (Data Layer) disimpan di dalam sebuah variabel array state JavaScript bernama dataMahasiswa. Tampilan tabel di layar peramban (View Layer) hanyalah representasi grafis visual sementara dari state tersebut. Setiap aksi penambahan data, penghapusan data, maupun penyaringan pencarian tidak boleh langsung memanipulasi baris

tabel HTML secara parsial; sebaliknya, skrip wajib memodifikasi array state terlebih dahulu, lalu memanggil fungsi render terpusat renderTable() untuk merekonstruksi tampilan DOM secara serentak.

Manipulasi array dilakukan menggunakan metode bawaan JavaScript: metode mutasi seperti .push() menambahkan objek baru ke akhir array, sedangkan .splice(index, 1) menghapus satu elemen pada indeks tertentu berdasarkan hasil pencarian .findIndex(). Untuk fitur pencarian langsung (Live Search), skrip memanfaatkan metode imutabel .filter() yang dieksekusi secara asinkron setiap kali pengguna mengetikkan karakter pada kotak input (<input id="input-search">). Metode .filter() mengevaluasi setiap objek mahasiswa: jika nilai NPM atau Nama mengandung penggalan kata kunci pencarian (diperiksa menggunakan String.prototype.includes() setelah diseragamkan dengan .toLowerCase()), objek tersebut dimasukkan ke dalam array hasil filter untuk segera dirender ke layar, sekaligus memperbarui counter total data secara instan.

### 1.11 Kamus Metode & Properti JavaScript DOM Penting untuk Judul 3

**Tabel 7 Kamus Metode & Properti JavaScript DOM Judul 3:** Ringkasan metode dan properti kunci yang dipelajari dan diterapkan pada Percobaan 3:

| Kategori | Metode / Properti | Tipe / Sintaks | Fungsi & Peran Utama |
|---|---|---|---|
| Selektor DOM | document.getElementById(id) | Metode Elemen | Menemukan satu node elemen berdasarkan atribut ID uniknya. |
| Manipulasi Konten | element.textContent | Properti String | Membaca atau menulis teks murni secara aman tanpa parsing HTML. |
| Manipulasi Konten | element.innerHTML | Properti String | Membaca atau menyisipkan baris kode HTML lengkap ke dalam elemen. |

| | | | |
|---|---|---|---|
| Manipulasi Kelas | element.classList.add / remove | Metode Properti | Menambahkan atau mencabut kelas CSS visual secara dinamis. |
| Manipulasi Node | document.createElement(tag) | Metode Factory | Membuat objek node elemen HTML baru di dalam memori peramban. |
| Manipulasi Node | parent.appendChild(node) | Metode Pohon | Menyisipkan node anak baru ke urutan terbawah dari elemen induk. |
| Peristiwa (Events) | element.addEventListener(ev, fn) | Metode Registrasi | Mendaftarkan fungsi penangan peristiwa (event listener) pada elemen. |
| Kontrol Peristiwa | event.preventDefault() | Metode Objek Event | Mencegah perilaku default browser (seperti reload saat form submit). |
| Kotak Dialog | window.confirm(pesan) | Metode Dialog | Menampilkan popup konfirmasi dengan tombol OK (true) dan Batal (false). |
| Pewaktu (Timer) | window.setTimeout(fn, ms) | Metode Web API | Mengeksekusi fungsi callback sekali setelah jeda waktu milidetik tertentu. |
| Metode Array | array.push(obj) | Metode Mutasi | Menambahkan objek baru ke posisi indeks paling belakang dari array. |

| | | | |
|---|---|---|---|
| Metode Array | array.splice(idx, jml) | Metode Mutasi | Menghapus sejumlah elemen dari array mulai dari indeks tertentu. |
| Metode Array | array.filter(callback) | Metode Imutabel | Menghasilkan array baru berisi elemen-elemen yang lolos kondisi evaluasi. |
| Metode Array | array.some(callback) | Metode Boolean | Mengembalikan true jika minimal ada satu elemen yang memenuhi syarat kondisi. |
| Metode Array | array.findIndex(callback) | Metode Numerik | Mencari nomor indeks elemen pertama yang memenuhi kriteria pencarian. |